

Documentación Proyecto NexoZone:

1a Parte (Propuesta del proyecto y Introducción del mismo):

1. Introducción y objetivos

Nexo Zone es una red social dedicada al mundo de los videojuegos, creada para reunir a jugadores con intereses similares.

El objetivo principal es ofrecer un espacio digital donde los usuarios puedan:

- Publicar opiniones y reseñas de videojuegos.
- Conocer personas con gustos parecidos.
- Consultar noticias y novedades del sector.
- Compartir contenido divertido (memes, clips, fanarts, etc.).
- Crear y compartir mods o fangames.
- Competir en minijuegos y rankings integrados dentro de la aplicación.

Además, la plataforma cuenta con una tienda virtual donde los usuarios pueden vender o compartir sus creaciones (mods, fangames, skins, etc.) mediante contacto privado y sistema de pago mediante Bizum.

2. Análisis del sector

Existen múltiples plataformas sociales generalistas como Instagram, Facebook o TikTok, pero ninguna se centra exclusivamente en los videojuegos y sus comunidades.

Algunas aplicaciones que se acercan a nuestra propuesta son:

- Steam Community – Espacio de noticias, foros y reseñas, pero limitado a títulos distribuidos en Steam.
- Game Jolt – Comunidad para creadores de juegos indie, con enfoque más técnico.
- Discord – Comunicación entre jugadores, pero sin estructura de red social ni contenido público por categorías.
- Reddit (r/gaming) – Foro global sobre videojuegos, pero sin identidad visual ni perfiles personalizables.

Nexo Zone busca combinar lo mejor de estas plataformas:

una red social dinámica, con comunicación directa entre usuarios, categorías de juegos, marketplace integrado, minijuegos, rankings y espacios para reseñas y debates.

3. Necesidades de los usuarios

Los usuarios de Nexo Zone necesitan:

- Un entorno social seguro y accesible para conocer jugadores con intereses comunes.
 - Publicar y consultar reseñas y opiniones de videojuegos.
 - Buscar juegos por nombre.
 - Poder comprar mods y fangames de otros usuarios.
 - Disponer de un sistema de mensajería privada en tiempo real.
 - Contar con opciones de personalización del perfil (biografía, imagen...).
 - Tener un sistema de notificaciones para interacciones relevantes.
 - Acceder fácilmente desde distintos dispositivos sin iniciar sesión cada vez.
 - Realizar ventas de contenido digital.
 - Participar en rankings y minijuegos para aumentar la interacción dentro de la plataforma.
-

4. Oportunidades y Amenazas

Oportunidades

- Alta demanda de espacios específicos para gamers.
- Creciente interés por los mods y juegos fanmade.
- Posibilidad de promoción cruzada en otras redes (TikTok, Instagram, Twitch).
- Comunidad en expansión de desarrolladores indie.
- Integración de notificaciones push y tiempo real mediante Firebase.
- Incorporación de minijuegos y rankings para aumentar la retención de usuarios.

Amenazas

- Competencia de grandes plataformas (Discord, Reddit, Steam).
 - Dificultad inicial para ganar visibilidad frente a marcas consolidadas.
 - Riesgos de seguridad (protección de datos y cuentas).
 - Posibles problemas de copyright con contenido fanmade.
-

5. Roles de usuario

Se definen dos roles principales:

Usuario estándar

- Crear una cuenta (correo o Google).
- Personalizar su perfil.
- Publicar reseñas, opiniones o memes.
- Interactuar (likes, comentarios...).
- Buscar juegos o usuarios.
- Enviar mensajes privados en tiempo real.
- Subir y vender mods o fangames.
- Participar en rankings y minijuegos.

Administrador

- Revisar y moderar publicaciones o denuncias.
 - Gestionar usuarios (bloqueos, permisos).
 - Controlar categorías de juegos y etiquetas.
 - Resolver incidencias técnicas o de soporte.
 - Gestionar solicitudes de etiquetas y moderación general de la comunidad.
-

6. Funcionalidades agrupadas por rol

Usuario estándar

- Registro e inicio de sesión (Google, correo electrónico).
- Edición de perfil: imagen, biografía.
- Publicaciones: texto e imágenes.
- Categorías: asignar etiquetas o categorías a publicaciones.
- Comentarios y likes en publicaciones de otros usuarios.
- Chat privado en tiempo real con otros usuarios.
- Sistema de notificaciones internas y push.
- Búsqueda por filtros:
 - Por usuario
 - Por minijuego
- Marketplace integrado:
 - Subir mods o fangames.
 - Publicaciones con información de venta.
- Rankings y minijuegos integrados.

Administrador

- Panel de control con gestión de usuarios y publicaciones.
 - Gestión de categorías y etiquetas.
 - Moderación de solicitudes de etiquetas.
 - Estadísticas generales de actividad.
-

7. Información detallada del sistema

Usuarios

- ID único, nombre, email y contraseña encriptada.
- Perfil con imagen, biografía.
- Tokens de notificaciones y sistema de administración.

Juegos y etiquetas

- Categorías y etiquetas personalizadas.
- Sistema de solicitudes de nuevas etiquetas por parte de usuarios.

Publicaciones

- ID, usuario autor, fecha, descripción, imágenes, categoría, likes y comentarios.
- Posibilidad de convertir publicaciones en productos en venta.

Chats

- Sistema de mensajería privada en tiempo real.
- Chats almacenados mediante colecciones separadas para mensajes y conversaciones recientes.
- Sistema de mensajes leídos y ocultación individual de chats.

Notificaciones

- Notificaciones internas dentro de la app.
- Notificaciones push mediante Firebase Cloud Messaging.

Rankings y minijuegos

- Sistema de puntuaciones y rankings por juego.
 - Clasificaciones almacenadas en Firebase Firestore.
-

8. Requerimientos del sistema

Funcionales

- Registro e inicio de sesión con Google o correo.
- Publicación y categorización de contenido.
- Sistema de comentarios y likes.
- Búsqueda avanzada.
- Sistema de mensajería privada en tiempo real.
- Marketplace para productos digitales.
- Moderación de contenido.
- Sistema de notificaciones.
- Rankings y minijuegos.

No funcionales

- Tiempo de carga reducido y sincronización en tiempo real.
 - Contraseñas encriptadas y almacenamiento seguro.
 - Interfaz intuitiva y accesible.
 - Compatibilidad con móviles Android.
 - Cumplimiento con la normativa de protección de datos (GDPR).
 - Arquitectura escalable basada en Firebase.
-

9. Tecnologías propuestas

- Firebase:
 - Authentication
 - Firestore Database
 - Firebase Cloud Messaging
 - Cloud Functions
 - Lenguaje principal: Kotlin.
 - Plataforma: Android Studio.
 - Backend serverless basado en Firebase.
 - Seguridad:
 - FirebaseAuth
 - Validaciones
 - Gestión de tokens FCM
 - Notificaciones push mediante Firebase Cloud Messaging.
 - Cloud Functions con Node.js para automatización de notificaciones.
-

10. Conclusión

Nexo Zone busca ser un punto de encuentro seguro, dinámico y creativo para los amantes de los videojuegos.

Combinando funciones sociales, marketplace, chat privado, rankings, minijuegos y notificaciones en tiempo real, la aplicación permite conectar personas, descubrir contenido y fomentar la creación de proyectos fanmade de forma accesible y moderna.

Gracias al uso de Firebase como backend serverless y al desarrollo en Kotlin para Android, la aplicación ofrece sincronización en tiempo real, escalabilidad y una arquitectura moderna sin necesidad de servidores tradicionales.

2a Parte (orientada a cómo funciona la aplicación):

11.Idea General

NexoZone usa Firebase como backend principal: autenticación, base de datos en tiempo real, notificaciones push, tokens de dispositivos y almacenamiento/descarga de archivos. La app Android está hecha en Kotlin y se comunica con Firebase directamente desde las Activities y varios managers propios.

La app no depende de un servidor tradicional para casi todo: Firebase actúa como backend serverless. Eso permite venderla como una aplicación escalable, rápida de mantener y con sincronización en tiempo real.

12.Transferencia De Datos

La app trabaja principalmente con Firestore. Los datos se envían como mapas Kotlin, normalmente `HashMapOf<String, Any>` o `mapOf<String, Any>`, por ejemplo en publicaciones, chats, comentarios, notificaciones y perfiles.

No se usa una serialización clásica tipo `Serializable`, `Parcelable` o JSON manual para guardar en Firebase. Las data class como `Post`, `Product`, `Chat`, `Message`, `Comment` o `NotificationItem` funcionan sobre todo como modelos internos de la app para pintar la interfaz.

Firestore serializa automáticamente valores simples:

`String`, `Int`, `Double`, `Boolean`, listas, mapas, `Timestamp`, y valores especiales como `FieldValue.increment()` o `FieldValue.arrayUnion()`.

La app normalmente hace:

- Escribir: `collection(...).add(hashMapOf(...))`
- Actualizar: `document(...).update(...)`
- Fusionar datos: `.set(data, SetOptions.merge())`
- Escuchar cambios en tiempo real: `.addSnapshotListener { ... }`
- Leer una vez: `.get().addOnSuccessListener { ... }`

Ejemplo importante: en ConversationActivity.kt los mensajes se guardan en: chats/{chatId}/messages

Y la lista lateral de chats se guarda aparte en: users/{uid}/recentChats/{otherUid}

Eso permite que el chat real siga existiendo aunque un usuario lo oculte de su lista.

13.Colecciones Principales

users: perfil, nombre visible, usuario único, bio, foto, tokens FCM, preferencias, admin, Bizum.

posts: publicaciones normales y productos en venta. Incluye authorUid, username, description, imageBase64, productName, productPrice, tag, likes, etc.

posts/{postId}/comments: comentarios de cada publicación.

chats/{chatId}/messages: mensajes privados entre dos usuarios.

users/{uid}/recentChats: resumen personalizado de chats para cada usuario.

users/{uid}/notifications: notificaciones internas de la app.

minigameScores/{gameType}/scores: rankings de minijuegos.

tags y tagRequests: sistema de etiquetas y solicitudes.

fcmDeviceTokens: registro seguro de tokens de notificaciones por dispositivo.

14.Serialización

Las clases no están serializadas como objetos completos hacia Firestore. La app construye documentos manualmente con campos concretos. Esto tiene una ventaja: el formato de datos queda muy controlado y es fácil evolucionar campos sin romper toda la app.

Por ejemplo, Post existe como data class, pero al publicar se crea un mapa manual en AddPostActivity.kt. Luego, al leer publicaciones, se reconstruye un Post leyendo campos como doc.getString(...), doc.getDouble(...), doc.getLong(...).

Para navegar entre pantallas, la app no serializa objetos completos: usa Intent.putExtra(...) con datos simples como ids, nombres, urls o descripciones.

15.Tiempo Real

Una parte fuerte de la app es el tiempo real. Usa addSnapshotListener para:

- Feed de publicaciones.
- Comentarios.
- Chats.
- Mensajes.
- Notificaciones.
- Rankings.

- Badges del menú inferior.

Esto permite decir en una presentación: “La aplicación no necesita refrescar manualmente; los cambios llegan automáticamente”.

16. Autenticación

La autenticación se basa en FirebaseAuth.

Hay inicio con email/contraseña y con Google. El botón de Google ahora queda solo en login y registro, no en la pantalla inicial.

Clase clave: `GoogleAuthManager.kt`

Puntos importantes:

- Valida configuración de Google.
- Usa `GoogleAuthProvider`.
- Si un usuario se registra con Google, se le pide crear contraseña.
- Vincula Google + contraseña con `linkWithCredential`.
- Si algo falla durante el registro Google, intenta eliminar la cuenta incompleta.

Clase clave de perfil: `AuthProfileManager.kt`

Esta clase asegura que cada usuario tenga documento en users, con: uid, email, username, displayName, privateId, bio, foto, preferencias y admin.

17. Notificaciones

Hay dos niveles de notificaciones.

Dentro de la app:

`users/{uid}/notifications`

Push al móvil:

Firebase Cloud Messaging + Cloud Function en `functions/index.js`

La app guarda tokens con `NotificationTokenManager.kt`. Cada dispositivo tiene token FCM y se registra tanto en el usuario como en `fcmDeviceTokens`.

Esto permite:

- Enviar push solo a dispositivos reales del usuario.
- Eliminar tokens antiguos.
- Respetar preferencias de notificación.

La Cloud Function escucha cuando se crea o modifica una notificación y manda el push. Además resuelve nombres actualizados del remitente para mensajes.

18. Chats

El chat tiene una arquitectura bastante buena para explicar:

- El mensaje real vive en `chats/{chatId}/messages`.

- Cada usuario tiene su resumen en recentChats.
- El resumen se puede ocultar solo para un usuario.
- El chat no se borra para la otra persona.
- El doble tick depende del campo isRead.
- El mensaje solo se marca como leído cuando la otra persona entra realmente al chat.

Eso es vendible como privacidad y control individual: cada usuario gestiona su bandeja sin afectar a la conversación global.

19.Compras Y Ventas

La app permite convertir una publicación en venta. Los productos se guardan dentro de posts usando campos como:
productName, productPrice, sellerBizumNumber, saleMode, zipFileName, zipDownloadUrl.

La pantalla de compras consulta publicaciones que tienen producto. Es un modelo simple: publicación social + producto en una misma entidad.

20.Minijuegos

Los minijuegos usan rankings en Firestore. Cada juego tiene scores por usuario. Esto crea retención: el usuario no solo publica o compra, también compete y vuelve a la app.

21.Puntos Para Vender

NexoZone mezcla comunidad, marketplace, chat privado, minijuegos, rankings y notificaciones en una sola plataforma.

Frases útiles:

- “No es solo una red social: es una comunidad interactiva con comercio, mensajería y gamificación.”
- “La app está construida sobre Firebase, lo que permite sincronización en tiempo real y escalabilidad sin montar servidores propios.”
- “Cada usuario tiene una identidad persistente, perfil editable, chats privados y notificaciones personalizadas.”
- “El sistema de rankings y minijuegos aumenta la retención.”
- “La arquitectura separa los datos reales del chat de la vista personalizada de cada usuario, permitiendo ocultar chats sin borrar conversaciones.”
- “La app usa notificaciones push y preferencias para que el usuario controle qué quiere recibir.”
- “El sistema de etiquetas y solicitudes permite que la comunidad crezca sin perder moderación.”

22.Cosas Técnicas A Destacar

Usa SetOptions.merge() para actualizar sin destruir documentos completos.

Usa batch para operaciones múltiples, por ejemplo tokens o propagación de nombres.

Usa runTransaction en compras para confirmar pagos evitando estados inconsistentes.

Usa FieldValue.increment() para contadores como no leídos.

Usa `FieldValue.arrayUnion()` y `arrayRemove()` para likes.

Usa `Timestamp.now()` para ordenar datos cronológicamente.

Usa `collectionGroup("comments")` y `collectionGroup("notifications")` para actualizar datos repartidos en subcolecciones.

23.Detalle Importante

Las imágenes se están guardando muchas veces como Base64 en Firestore. Para una demo está bien porque simplifica, pero para vender una versión más profesional se puede decir que el siguiente paso natural sería mover imágenes grandes a Firebase Storage y dejar en Firestore solo la URL.